

Introdução à C/C++ para programadores Python - Parte 2

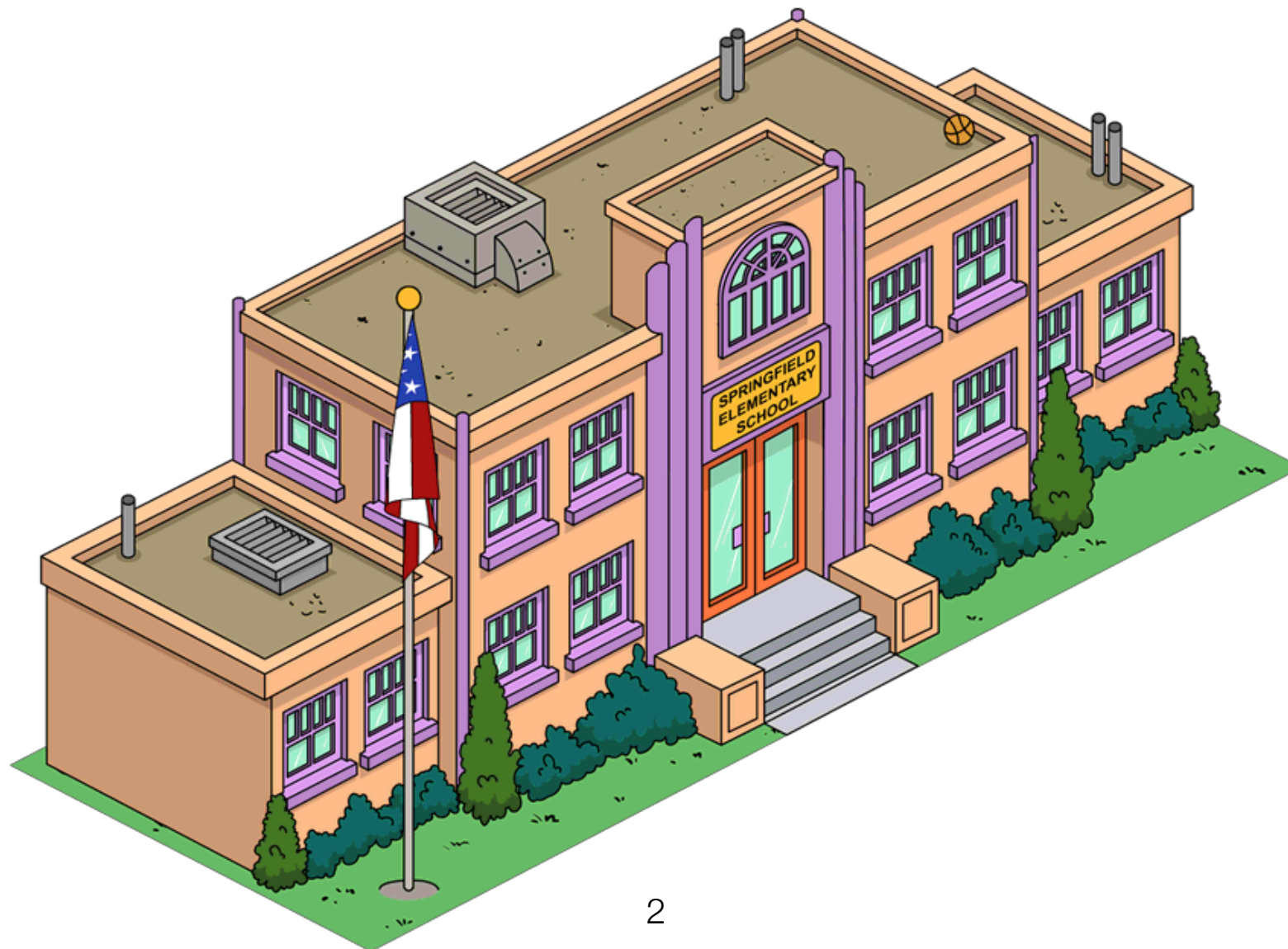
Algoritmos e Programação 2

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

Problema: Média do Colégio

Faça um algoritmo que leia todas as notas de um colégio inteiro e verifique se a média do colégio está acima de 7.
Obs.: Suponha que existam 200 estudantes.



Exemplo 1: Média do Colégio

```
1. #include <stdio.h>
2.
3. int main () {
4.     int i;
5.     float notas[200], media = 0;
6.
7.     printf("Entre com as notas: ");
8.     for (i = 0; i < 200; i++) {
9.         scanf("%f", &notas[i]);
10.        media += notas[i];
11.    }
12.    media = media / 200;
13.
14.    printf("media = %f\n", media);
15.    if (media >= 7.0)
16.        printf("Media da escola maior que 7.\n");
17.    else
18.        printf("Media da escola menor que 7.!\n");
19.
20.    return 0;
21.}
```

Vetores

```
int notas[200];
```

- Conjunto de variáveis do **mesmo tipo** e **mesmo identificador** (nome)
 - São **alocadas sequencialmente** na memória
- O que as distingue é um **índice** (0 a $n - 1$) que referencia sua localização dentro da estrutura

Exemplo 2: Encontre o Maior Elemento

Faça um programa que declare um vetor de inteiros de tamanho 10, leia 10 valores e armazene-os no vetor. Após isso, encontre e imprima o maior elemento do vetor.

Exemplo 2: Encontre o Maior Elemento (Cont.)

```
1. #include <stdio.h>
2. #define TAMANHO 10
3.
4. int main () {
5.     int v[TAMANHO], i, maior;
6.
7.     printf("Entre com os valores: ");
8.     for(i = 0; i < TAMANHO; i++) {
9.         scanf("%d", &v[i]);
10.    }
11.
12.    maior = v[0];
13.    for(i = 1; i < TAMANHO; i++) {
14.        if(maior < v[i]) {
15.            maior = v[i];
16.        }
17.    }
18.    printf("\nmaior = %d\n", maior);
19.
20.    return 0;
21.}
```

Matrizes em C/C++

- Considerando-se o problema de ler 3 notas de uma turma de 8 alunos

```
int notas[3][8];
```

	0	1	2	3	4	5	6	7
0								
1								
2								

```
int notas[8][3];
```

	0	1	2
0			
1			
2			
3			
4			
5			
6			
7			

Preenchendo uma Matriz

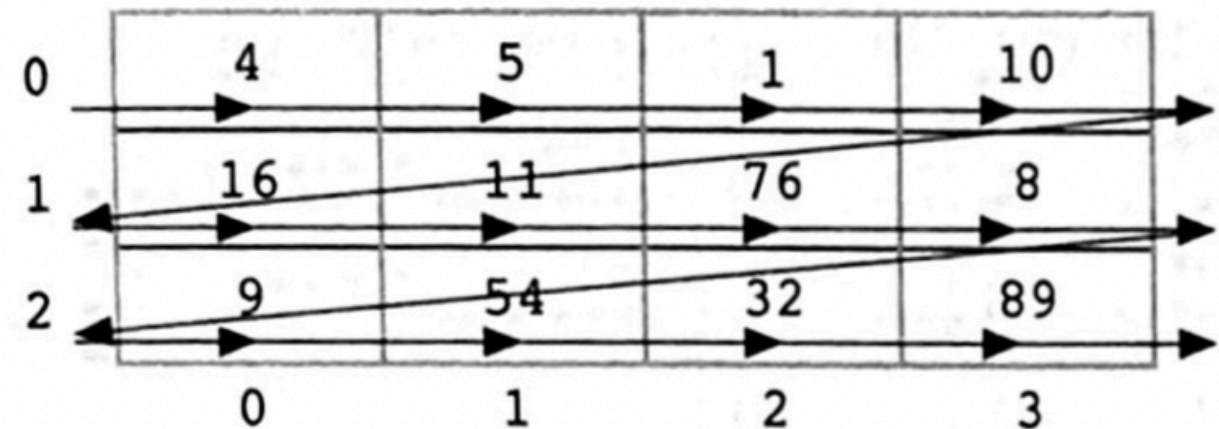
Exemplo leitura das notas de cada prova para cada aluno:

```
for (i = 0; i < 8; i++) {  
    for (j = 0; j < 3; i++) {  
        printf("Digite a nota da prova %d do aluno %d", j, i);  
        scanf("%d", &notas[i][j]);  
    }  
}
```


Percorrendo uma Matriz: Por Linha

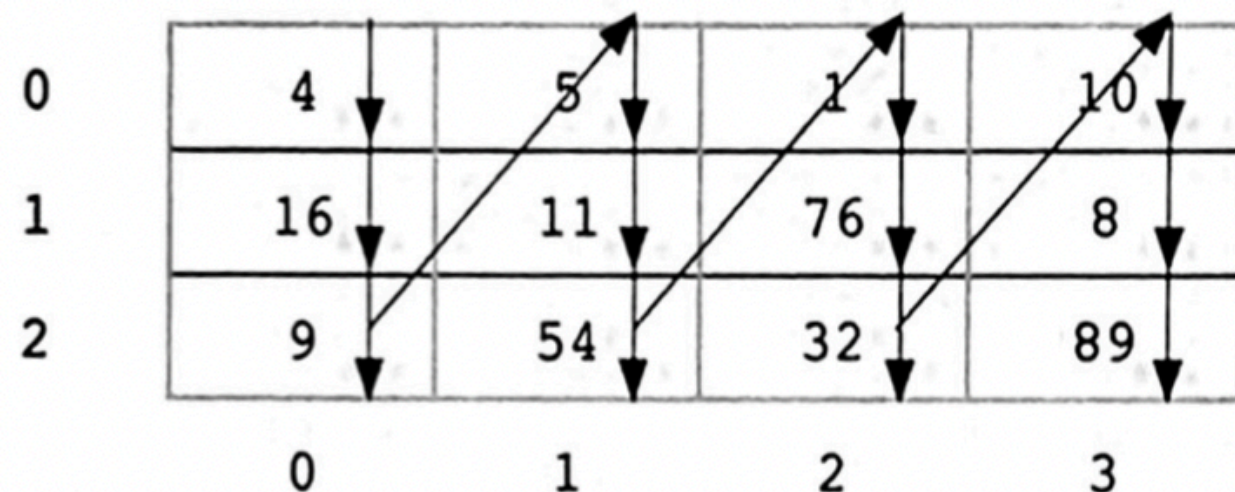
- **Forma 1:** Mostrar elementos **por linha**

```
for (i = 0; i < 3; i++) {  
    printf("Elementos da linha %d", i);  
    for (j = 0; j < 4; j++)  
        printf("%d", x[i][j])  
}
```



- **Forma 2:** Mostra elementos por **coluna**

```
for (i = 0; i < 4; i++) {  
    printf("Elementos da coluna %d", i)  
    for (j = 0; j < 3; j++)  
        printf("%d", x[j][i])  
}
```



Modularização

Funções

- Princípio **programação procedural** é dividir um programa em funções
 - Dividem grandes tarefas em tarefas menores
- Reusabilidade

Exemplo 3: Fahrenheit para Celsius

```
1. #include <stdio.h>
2.
3. float converte_fahr_para_celsius(float fahr);
4.
5. int main () {
6.     float c, f;
7.
8.     printf("Digite a temperatura em graus Fahrenheit: ");
9.     scanf("%f", &f);
10.
11.     c = converte_fahr_para_celsius(f);
12.
13.     printf("Celsius = %.2f\n", c);
14.     return 0;
15. }
16.
17. float converte_fahr_para_celsius(float fahr) {
18.     float c;
19.     c = (fahr - 32.0) * 5.0/9.0;
20.     return c;
21. }
22.
```

Celsius para Fahrenheit:

$$T_F = \left(\frac{9}{5} T_C \right) + 32$$

Fahrenheit para Celsius:

$$T_C = \frac{5}{9} (T_F - 32)$$

Componentes de uma Função

1. Protótipo da função
2. Definição da função
3. Chamada à função

Protótipo da Função

- Precede definição e chamada
- Fornece ao compilador informações sobre **nome da função**, **tipo de retorno**, **número** e o **tipo dos argumentos**
- Sem o protótipo, o compilador não tem como verificar e checar se há erros em seu uso
- O nosso exemplo declara o seguinte protótipo:

```
float converte_fahr_para_celsius(float fahr);
```

Tipo de uma Função

- É definido pelo tipo de valor que ela retorna por meio do comando **return**
- Uma função é do tipo **float** quando retorna um valor do tipo **float**
- Os tipos de funções são os mesmos tipos das variáveis, exceto quando a função não retorna nada. Nesse caso, ela é do tipo **void**

Parâmetros de uma Função

```
float converte_fahr_para_celsius(float fahr) {  
    float c;  
    c = (fahr - 32.0) * 5.0/9.0;  
    return c;  
}
```

- No trecho acima, é definido apenas um parâmetro chamado *fahr* que é do tipo **float**

Passagem de Argumentos por
Valor x por Referência

Passagem por Valor

- Em nosso exemplo, a função cria uma nova variável para receber o valor passado
- Sua declaração indica que o valor enviado será armazenado na variável *fahr*, criada quando a função inicia sua execução e destruída quando ela termina

Exemplo 4: a Antes e Depois

```
1. #include <stdio.h>
2.
3. void f(int a);
4.
5. int main () {
6.     int a;
7.     a = 2;
8.
9.     printf("a antes = %d\n", a);
10.
11.    f(a);
12.
13.    printf("a depois = %d\n", a);
14.
15.    return 0;
16.}
17.
18.void f(int a) {
19.    a += 1;
20.}
```

```
% ./a.out
a antes = 2
a depois = 2
```

Exemplo 5: b[] Antes e Depois

```
1. #include <stdio.h>
2.
3. void f(int b[]);
4.
5. int main () {
6.     int b[2] = {2, 10};
7.
8.     printf("antes\nb[0] = %d, b[1] = %d\n\n", b[0], b[1]);
9.     f(b);
10.    printf("depois\nb[0] = %d, b[1] = %d\n", b[0], b[1]);
11.
12.    return 0;
13.}
14.
15. void f(int b[]) {
16.     b[0] += 1;
17.     b[1] += 1;
18.}
```

```
% ./a.out
antes
b[0] = 2, b[1] = 10

depois
b[0] = 3, b[1] = 11
```

Passagem por Valor x Referência

- Por enquanto precisamos saber somente que:
 - **Passagem por valor:** Para as variáveis, exceto vetores, qualquer alteração do parâmetro dentro da função não irá causar nenhuma modificação fora da função
 - **Passagem por referência:** Para vetores, alterar o conteúdo do vetor dentro de uma função irá alterar também o conteúdo do argumento do vetor passado

Strings

Strings na Linguagem C/C++

- C/C++ não possui um tipo nativo de dado **string**
- Armazena uma cadeia de caracteres em um vetor
 - Cada posição representa um caractere
 - O fim de uma cadeia é identificado por meio do caractere nulo, ou seja, por meio do **'\0'**
- Deve-se declarar sempre o vetor com uma posição a mais para armazenar o caractere nulo

Exemplo 7: Cadeia

```
char palavra[7] = "CADEIA";
```

índice	...	0	1	2	3	4	5	6	...
valor	...	C	A	D	E	I	A	\0	...
posição memória	...	863	864	865	866	867	868	869	...

Inicializando Cadeias de Caracteres

- Inicialização no momento da declaração

```
char nome[] = "Programa";
```

- Inicialização por meio da atribuição (depois da declaração)

```
char vet1[10], vet2[5] = "casa";  
strcpy(vet1, "Programa");  
strcpy(vet1, vet2);
```

Inicializando Cadeias de Caracteres (Cont.)

- Inicialização por meio do teclado

```
char vet[10];  
scanf("%s", vet);
```

Obs.: Armazena todos os símbolos digitados até a ocorrência do primeiro espaço em branco

ou

```
fgets(vet, 10, stdin);
```

Concatenando Cadeias de Caracteres

`strcat(str1, str2);`

- A função **strcat** concatena a cadeia de caracteres **str2** à **str1**

`strncat(str1, str2);`

- A função **strncat** concatena os n primeiros caracteres da cadeia **str2** à **str1**

Comparando Cadeias de Caracteres

```
resultado = strcmp(cadeia1, cadeia2);
```

- A função **strcmp** compara duas cadeias e retorna um número inteiro, que poderá ser:
 - 0 se cadeias forem iguais;
 - < 0 se **cadeia1** for alfabeticamente menor que a **cadeia2**;
 - > 0 se **cadeia1** for alfabeticamente maior que a **cadeia2**;
- Essa função considera letras maiúsculas símbolos diferentes de letras minúsculas

Exemplo 7: Strcmp

```
1. #include <stdio.h>
2. #include <string.h>
3.
4. int main () {
5.     char cor[10];
6.
7.     printf("Entre com uma cor: ");
8.     scanf("%s%c", cor);
9.
10.    if(strcmp(cor, "verde") == 0) {
11.        printf("Cor do melhor time do mundo!\n");
12.    }
13.
14.    printf("A cor escolhida e %s!\n", cor);
15.
16.    return 0;
17.}
18.
```

Tamanho da Cadeia de Caracteres

```
tamanho = strlen(cadeia);
```

- A função **strlen** retorna para a variável **tamanho** o número de caracteres da **cadeia**
- O caractere que indica final da cadeia de caracteres, **'\0'**, não entra no total de caracteres da cadeia

Referências

- ASCENCIO, Ana Fernanda Gomes; DE CAMPOS, Edilene Aparecida Veneruchi. Fundamentos da programação de computadores. 3.ed. Pearson Education do Brasil, 2012.
- DEITEL, Paul; DEITEL, Harvey; C++ How to Program (9 edition). Pearson, 2016.